

Threat Modeling Report

Created on 17-Mar-26 1:34:06 PM

Threat Model Name:

Owner:

Reviewer:

Contributors:

Description:

Assumptions:

External Dependencies:

Threat Model Summary:

Not Started	51
Not Applicable	0
Needs Investigation	5
Mitigation Implemented	0
Total	56
Total Migrated	0

Diagram: Diagram 1

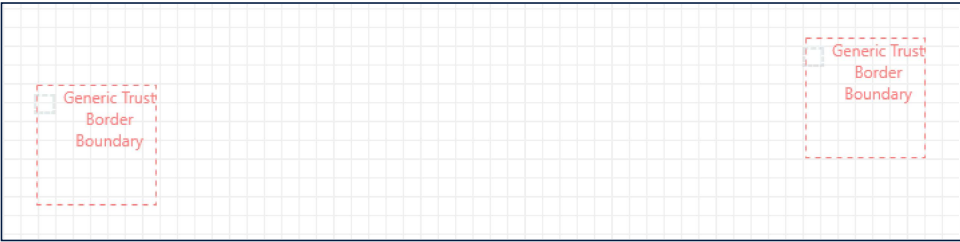


Diagram 1 Diagram Summary:

Not Started	0
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	0
Total Migrated	0

Diagram: Diagram 2

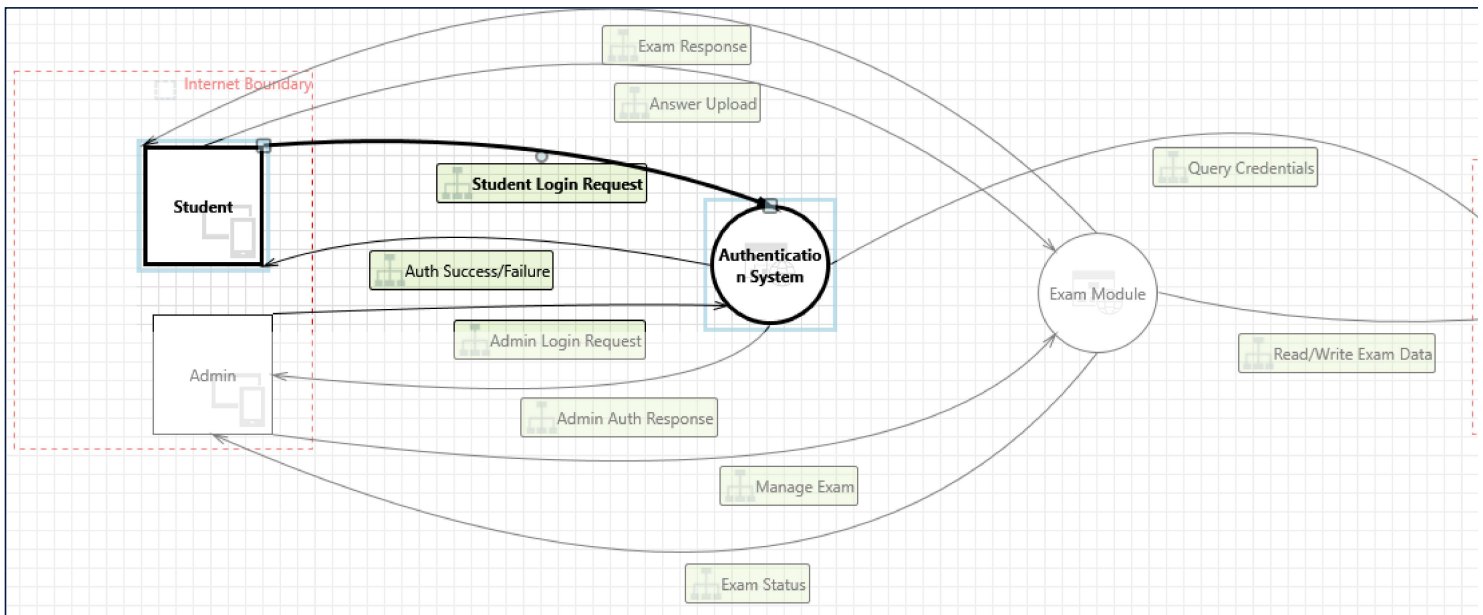
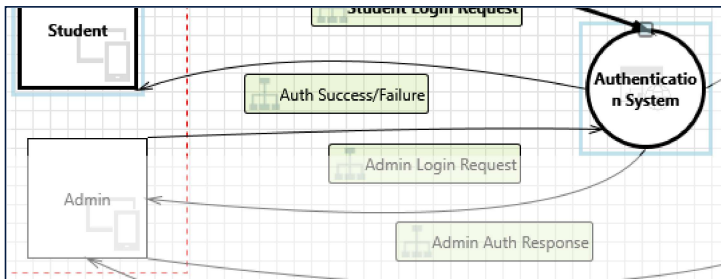


Diagram 2 Diagram Summary:

Not Started	51
Not Applicable	0
Needs Investigation	5
Mitigation Implemented	0
Total	56
Total Migrated	0

Interaction: Admin Login Request



1. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASPNET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

2. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdata#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtdata#autocomplete-input

href="https://aka.ms/tmtinputval#typemodel">https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

3. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing
Description: Ensure that TLS certificate parameters are configured with correct values
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts
SDL Phase: Implementation

4. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation
Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management
SDL Phase: Implementation

5. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details
Justification: <no mitigation provided>
Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASPNET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum
SDL Phase: Implementation

6. An adversary can gain sensitive data from mobile device [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: If application saves sensitive PII or HBI data on phone SD card or local storage, then it may get stolen.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sensitive or PII data written to phones local storage. Refer: https://aka.ms/tmtdata#pii-phones
SDL Phase: Implementation

7. An adversary can gain access to sensitive data by sniffing traffic from Mobile client [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data by sniffing traffic from Mobile client
Justification: <no mitigation provided>
Possible Mitigation(s): Implement Certificate Pinning. Refer: https://aka.ms/tmtcommsec#cert-pinning

SDL Phase: Implementation

8. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

9. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

10. An adversary may jail break into a mobile device and gain elevated privileges [State: Not Started] [Priority: High]

Category: Elevation of Privileges

Description: An adversary may jail break into a mobile device and gain elevated privileges

Justification: <no mitigation provided>

Possible Mitigation(s): Implement implicit jailbreak or rooting detection. Refer: https://aka.ms/tmtauthz#rooting-detection

SDL Phase: Design

11. An adversary may spoof Admin and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

12. An adversary can reverse engineer and tamper binaries [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can use various tools, reverse engineer binaries and abuse them by tampering

Justification: <no mitigation provided>

Possible Mitigation(s): Obfuscate generated binaries before distributing to end users. Refer: https://aka.ms/tmtdata#binaries-end

SDL Phase: Design

13. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

14. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

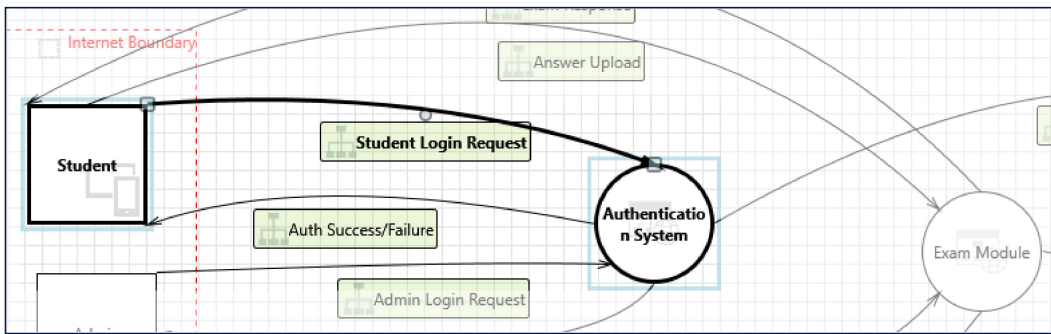
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

Interaction: Answer Upload



15. An adversary may jail break into a mobile device and gain elevated privileges [State: Not Started] [Priority: High]

Category: Elevation of Privileges

Description: An adversary may jail break into a mobile device and gain elevated privileges

Justification: <no mitigation provided>

Possible Mitigation(s): Implement implicit jailbreak or rooting detection. Refer: https://aka.ms/tmtauthz#rooting-detection

SDL Phase: Design

16. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

17. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary may gain access to sensitive data from log files
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access
SDL Phase: Implementation

18. An adversary can gain access to sensitive data by sniffing traffic from Mobile client [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data by sniffing traffic from Mobile client
Justification: <no mitigation provided>
Possible Mitigation(s): Implement Certificate Pinning. Refer: https://aka.ms/tmtcommsec#cert-pinning
SDL Phase: Implementation

19. An adversary can gain sensitive data from mobile device [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: If application saves sensitive PII or HBI data on phone SD card or local storage, then it ay get stolen.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sensitive or PII data written to phones local storage. Refer: https://aka.ms/tmtdata#pii-phones
SDL Phase: Implementation

20. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details
Justification: <no mitigation provided>
Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum
SDL Phase: Implementation

21. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation
Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management
SDL Phase: Implementation

22. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing
Description: Ensure that TLS certificate parameters are configured with correct values
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

23. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdat#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtdat#inputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtdat#inputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtdat#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtdat#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtdat#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtdat#string-method

SDL Phase: Implementation

24. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtdat#x509-sslts Ensure that authenticated ASPNET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtdat#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtdat#redirect-safe

SDL Phase: Implementation

25. An adversary may spoof Student and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtdat#standard-authn-web-app

SDL Phase: Design

26. An adversary can reverse engineer and tamper binaries [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can use various tools, reverse engineer binaries and abuse them by tampering

Justification: <no mitigation provided>

Possible Mitigation(s): Obfuscate generated binaries before distributing to end users. Refer: https://aka.ms/tmtdat#binaries-end

SDL Phase: Design

27. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtdat#typesafe

SDL Phase: Implementation

28. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

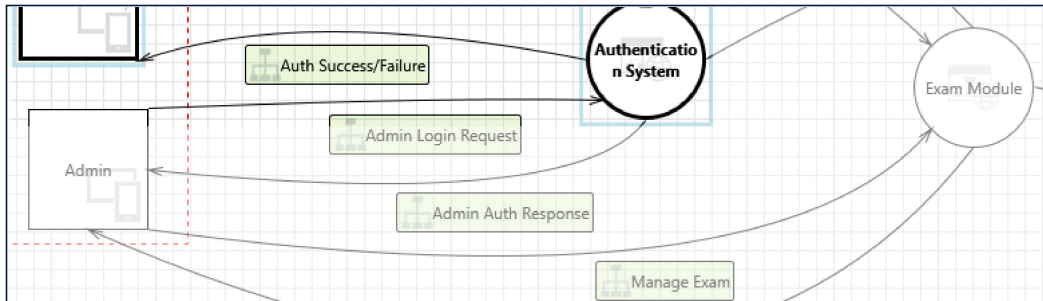
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

Interaction: Manage Exam



29. An adversary may jail break into a mobile device and gain elevated privileges [State: Not Started] [Priority: High]

Category: Elevation of Privileges

Description: An adversary may jail break into a mobile device and gain elevated privileges

Justification: <no mitigation provided>

Possible Mitigation(s): Implement implicit jailbreak or rooting detection. Refer: https://aka.ms/tmtauthz#rooting-detection

SDL Phase: Design

30. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

31. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

32. An adversary can gain access to sensitive data by sniffing traffic from Mobile client [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data by sniffing traffic from Mobile client
Justification: <no mitigation provided>
Possible Mitigation(s): Implement Certificate Pinning. Refer: https://aka.ms/tmtcommsec#cert-pinning
SDL Phase: Implementation

33. An adversary can gain sensitive data from mobile device [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: If application saves sensitive PII or HBI data on phone SD card or local storage, then it ay get stolen.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sensitive or PII data written to phones local storage. Refer: https://aka.ms/tmtdata#pii-phones
SDL Phase: Implementation

34. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details
Justification: <no mitigation provided>
Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum
SDL Phase: Implementation

35. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation
Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management
SDL Phase: Implementation

36. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing
Description: Ensure that TLS certificate parameters are configured with correct values
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts
SDL Phase: Implementation

37. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing
Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,
Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: <https://aka.ms/tmtdata#autocomplete-input> Perform input validation and filtering on all string type Model properties. Refer: <https://aka.ms/tmtdata#autocomplete-input> Validate all redirects within the application are closed or done safely. Refer: <https://aka.ms/tmtdata#autocomplete-input> Enable step up or adaptive authentication. Refer: <https://aka.ms/tmtdata#autocomplete-input> Implement forgot password functionalities securely. Refer: <https://aka.ms/tmtdata#autocomplete-input> Ensure that password and account policy are implemented. Refer: <https://aka.ms/tmtdata#autocomplete-input> Implement input validation on all string type parameters accepted by Controller methods. Refer: <https://aka.ms/tmtdata#autocomplete-input>

SDL Phase: Implementation

38. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts> Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: <https://aka.ms/tmtcommsec#x509-sslts> Validate all redirects within the application are closed or done safely. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

39. An adversary may spoof Admin and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: <https://aka.ms/tmtauthn#standard-authn-web-app>

SDL Phase: Design

40. An adversary can reverse engineer and tamper binaries [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can use various tools, reverse engineer binaries and abuse them by tampering

Justification: <no mitigation provided>

Possible Mitigation(s): Obfuscate generated binaries before distributing to end users. Refer: <https://aka.ms/tmtdata#binaries-end>

SDL Phase: Design

41. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: <https://aka.ms/tmtinputval#typesafe>

SDL Phase: Implementation

42. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

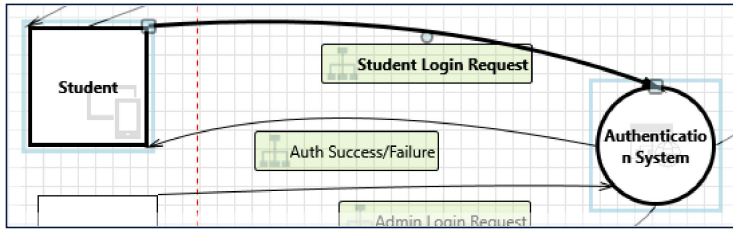
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: <https://aka.ms/tmtdata#encrypt-data>

SDL Phase: Implementation

Interaction: Student Login Request



43. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data
SDL Phase: Implementation

44. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering
Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tminputval#typesafe
SDL Phase: Implementation

45. An adversary can reverse engineer and tamper binaries [State: Needs Investigation] [Priority: High]

Category: Tampering
Description: An adversary can use various tools, reverse engineer binaries and abuse them by tampering
Justification: An adversary could tamper with the student login request data in transit by modifying authentication tokens or credentials if TLS encryption is not enforced on the channel.
Possible Mitigation(s): Obfuscate generated binaries before distributing to end users. Refer: https://aka.ms/tmtdata#binaries-end
SDL Phase: Design

46. An adversary may spoof Student and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing
Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application
Justification: <no mitigation provided>
Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app
SDL Phase: Design

47. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing
Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASPNET pages incorporate UI Redressing or clickjacking defences.

Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

48. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdata#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

49. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Needs Investigation] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: An adversary could spoof a legitimate student's identity by forging login credentials to gain unauthorized access to the exam system, bypassing authentication controls.

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

50. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Needs Investigation] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: Without proper audit logging on login attempts, a student could deny having accessed the exam system or submitted answers, making it impossible to trace malicious activity.

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

51. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASPNET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

52. An adversary can gain sensitive data from mobile device [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: If application saves sensitive PII or HBI data on phone SD card or local storage, then it ay get stolen.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sensitive or PII data written to phones local storage. Refer: https://aka.ms/tmtdata#pii-phones
SDL Phase: Implementation

53. An adversary can gain access to sensitive data by sniffing traffic from Mobile client [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data by sniffing traffic from Mobile client
Justification: <no mitigation provided>
Possible Mitigation(s): Implement Certificate Pinning. Refer: https://aka.ms/tmtcommsec#cert-pinning
SDL Phase: Implementation

54. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary may gain access to sensitive data from log files
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access
SDL Phase: Implementation

55. An adversary can reverse weakly encrypted or hashed content [State: Needs Investigation] [Priority: High]

Category: Information Disclosure
Description: An adversary can reverse weakly encrypted or hashed content
Justification: An adversary could intercept sensitive student login credentials flowing to the Authentication System if the data channel is not properly encrypted with TLS/HTTPS.
Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts
SDL Phase: Implementation

56. An adversary may jail break into a mobile device and gain elevated privileges [State: Needs Investigation] [Priority: High]

Category: Elevation of Privileges
Description: An adversary may jail break into a mobile device and gain elevated privileges
Justification: An adversary could flood the Authentication System with excessive student login requests, exhausting server resources and preventing legitimate students from accessing the exam system.
Possible Mitigation(s): Implement implicit jailbreak or rooting detection. Refer: https://aka.ms/tmtauthz#rooting-detection
SDL Phase: Design